

Memory Color Pattern

Conversation log with prompts, responses and code edits

Project: **58_color_pattern** (Python / pygame) • Generated October 8, 2026

Turn	Your request	File changed	Net change
1	Fix sequence duplication after round 1	game/game_engine.py	1 line fixed
2	Faster playback each round + a distinct sound per pad	game/game_engine.py	Speed schedule, generated tones, two small bug fixes
3	Player-turn countdown timer + final review pass	game/game_engine.py	Timer + bar, timeout game over, round-start pause

Code blocks below are unified diffs: lines on a green background (+) were added and lines on a red background (–) were removed. Grey lines are surrounding context. Every diff is generated from the actual file versions, not retyped.

Turn 1 — Fix the duplicated pattern

YOU

I ran the game and found that the pattern starts getting duplicated after the first round. Please go through the existing code and fix this issue so that each new round adds only one new random color to the existing sequence. Keep the current game structure and UI unchanged. Do not implement any other feature yet. After making the change, briefly explain what was wrong and what you changed.

Attached: 58_color_pattern.zip

CLAUDE — RESPONSE

The pattern now grows by exactly one random color per round. The fix is in `game/game_engine.py`, in `start_next_round()`.

What was wrong: the line was `self.sequence += self.sequence + [new_color]`. The `+=` already adds to the existing list, so the right-hand side (the whole sequence plus the new color) was added on top of the sequence again. That doubled the pattern each round: 1, 2, 4, 8, and so on.

What I changed: that line is now `self.sequence.append(new_color)`. Nothing else changed, so the game structure, UI and restart behavior are the same.

Check: a headless run where the player repeats the sequence correctly grew 1, 2, 3, 4, 5, with each earlier pattern kept as a prefix. After a game-over reset the sequence started again at length 1.

Code changes

game/game_engine.py	+2 -2
@@ -43,8 +43,8 @@	
def start_next_round(self):	
new_color = random.randint(0, 3)	
-	self.sequence += self.sequence + [new_color]
-	
+	self.sequence.append(new_color)
+	
self.player_input.clear()	
self.state = "WATCH"	
self.showing_step = 0	

Turn 2 — Faster playback and per-pad sounds

YOU

Now implement the next two features from the README. First, make the pattern playback gradually faster as the round number increases by reducing the flash and pause durations. It should still be easy to follow at the beginning and then become faster in later rounds.

Second, add a different sound or pitch for each of the four colored buttons. The sound should play both when the game shows a button during the pattern and when the player clicks that button.

Please keep the existing controls and UI and avoid adding unnecessary dependencies.

Test the logic for any obvious issues before finishing.

After making the changes, briefly explain what you changed.

CLAUDE — RESPONSE

Both features are in `game/game_engine.py`; nothing else changed. Controls, UI and dependencies are the same.

Playback acceleration

- Round 1 plays at the original speed (450 ms flash, 200 ms pause).
- Each later round removes 20 ms from the flash and 9 ms from the pause, so it speeds up gradually.
- It stops at 160 ms flash / 70 ms pause (around round 16), so the pattern stays followable.
- Timing is set at the start of each round; restarting with R returns to round 1 speed. The numbers are constants at the top of the file.

Sounds

- Each button gets its own pitch: Red C4 (262 Hz), Blue E4 (330 Hz), Green G4 (392 Hz), Yellow C5 (523 Hz).
- Tones are 0.2 s sine waves generated in code with the standard library: no audio files, no new packages.
- A tone plays when a pad is flashed during playback and when the player clicks one (including a wrong click).
- If no audio device is available, the game runs silently instead of crashing.

Two small bugs fixed along the way

- **Stuck-lit pad:** clicking two pads less than 150 ms apart left the first pad lit permanently. A pad is now switched off when the next one is clicked.

- **Click highlight overlap:** the last click's highlight could overlap or cut short the first flash of the next round. It was cleared when a round started (revisited in Turn 3).

Code changes

Diff shown against the Turn 1 result.

game/game_engine.py	+66 -2
@@ -1,6 +1,23 @@	
+import math	
import random	
+from array import array	
+	
import pygame	
from game.color_button import ColorButton	
+	
+	
+# Playback timing (milliseconds): starts slow and easy to follow, speeds up	
+# each round, and never goes below the minimums so it stays playable.	
+BASE_FLASH_MS = 450	
+BASE_PAUSE_MS = 200	
+MIN_FLASH_MS = 160	
+MIN_PAUSE_MS = 70	
+FLASH_STEP_MS = 20 # flash time removed per round after round 1	
+PAUSE_STEP_MS = 9 # pause time removed per round after round 1	
+	
+# One pitch (Hz) per button id: Red, Blue, Green, Yellow (C4, E4, G4, C5).	
+TONE_FREQUENCIES = [262, 330, 392, 523]	
+TONE_LENGTH_MS = 200	
class GameEngine:	
@@ -27,8 +44,8 @@	
self.state = "WATCH"	
self.showing_step = 0	
self.step_start_time = 0	
- self.flash_duration = 450	
- self.pause_duration = 200	
+ self.flash_duration = BASE_FLASH_MS	
+ self.pause_duration = BASE_PAUSE_MS	
self.is_flashing = False	
self.player_lit_button = None	
@@ -38,12 +55,54 @@	
self.font_title = pygame.font.SysFont(None, 40)	
self.font_medium = pygame.font.SysFont(None, 28)	
+ self.sounds = self.build_sounds()	
+	
self.start_next_round()	
+	
def build_sounds(self):	
"""Generate one short sine tone per button (no extra dependencies).	
Returns an empty list if audio is unavailable so the game still runs."""	
try:	
init = pygame.mixer.get_init()	
if init is None or init[1] != -16:	
pygame.mixer.quit()	
pygame.mixer.init(44100, -16, 1)	
sample_rate, _, channels = pygame.mixer.get_init()	
+	
sounds = []	
total = int(sample_rate * TONE_LENGTH_MS / 1000)	
fade = int(sample_rate * 0.015) # short fade in/out avoids clicks	
for freq in TONE_FREQUENCIES:	
samples = array("h")	
for i in range(total):	

```

+         envelope = min(1.0, i / fade, (total - i) / fade)
+         value = int(9000 * envelope * math.sin(2 * math.pi * freq * i / sample_rate))
+         samples.extend([value] * channels)
+         sounds.append(pygame.mixer.Sound(buffer=samples.tobytes()))
+     return sounds
+ except pygame.error:
+     return []
+
+ def play_tone(self, color_id):
+     if self.sounds:
+         self.sounds[color_id].play()
+
+ def update_playback_speed(self):
+     """Shorten flash and pause as the round number (sequence length) grows."""
+     rounds_played = len(self.sequence) - 1
+     self.flash_duration = max(MIN_FLASH_MS, BASE_FLASH_MS - FLASH_STEP_MS * rounds_played)
+     self.pause_duration = max(MIN_PAUSE_MS, BASE_PAUSE_MS - PAUSE_STEP_MS * rounds_played)
+
+ def start_next_round(self):
+     new_color = random.randint(0, 3)
+     self.sequence.append(new_color)
+     self.update_playback_speed()
+
+     # Drop any leftover click highlight so it can't overlap/cut the first flash
+     if self.player_lit_button is not None:
+         self.player_lit_button.is_lit = False
+         self.player_lit_button = None
+
+     self.player_input.clear()
+     self.state = "WATCH"
@@ -51,6 +110,7 @@
+     self.step_start_time = pygame.time.get_ticks()
+     self.is_flashing = True
+     self.buttons[self.sequence[0]].is_lit = True
+     self.play_tone(self.sequence[0])
+
+ def update(self):
+     now = pygame.time.get_ticks()
@@ -74,6 +134,7 @@
+         if self.showing_step < len(self.sequence):
+             next_id = self.sequence[self.showing_step]
+             self.buttons[next_id].is_lit = True
+             self.play_tone(next_id)
+             self.is_flashing = True
+             self.step_start_time = now
+         else:
@@ -88,9 +149,12 @@
+         if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
+             for btn in self.buttons:
+                 if btn.contains(event.pos):
+                     if self.player_lit_button is not None:
+                         self.player_lit_button.is_lit = False # avoid a stuck-lit pad on fast clicks
+
+                     btn.is_lit = True
+                     self.player_lit_button = btn
+                     self.player_lit_start = pygame.time.get_ticks()
+                     self.play_tone(btn.color_id)
+                     self.register_player_click(btn.color_id)
+                     break

```

Testing performed

- 20 simulated rounds with a fake clock: each round played the right tones in order, and playback time matched the flash/pause settings.
- Speed never drops below the minimums; a wrong click ends the game; R resets to round 1 speed.
- The no-audio case works.

- A test that clicked every 100 ms exposed the stuck-lit pad (a pre-existing issue), which led to the fix above.

Turn 3 — Player turn timer and final review

YOU

The last feature is the player turn timer from the README. Add a countdown timer/progress bar that is shown only during the player's turn. Give the player a reasonable amount of time to enter the current pattern, and make the allowed time depend on the pattern length so longer patterns get enough time. The timer should decrease while the player is entering the pattern and immediately trigger Game Over if it reaches zero before the pattern is completed. Reset the timer correctly whenever a new round starts or the game is restarted.

After implementing this, please do a final pass through the whole project for obvious runtime or logic issues introduced by these changes. In particular, check the WATCH/PLAYER_TURN/GAME_OVER state transitions, button lighting, timer reset, sound playback, sequence validation, and restarting with R. Fix any genuine issues you find, but don't unnecessarily rewrite working parts of the project. Keep the existing UI style.

Attached: `game_engine.py` (the version from Turn 2, with your added comment)

CLAUDE — RESPONSE

The player turn timer is added, and the final pass found one problem in my earlier work, which I fixed. All changes are in `game/game_engine.py`, built on the version you uploaded (your comment on the old bug line is kept).

The timer

- **Time allowed:** 3 s plus 1 s per step in the pattern, for the whole pattern: round 1 = 4 s, round 5 = 8 s, round 10 = 13 s.
- **How it counts:** starts when the player's turn begins and keeps running across clicks (a total, not per click).
- **Display:** a progress bar under the status line, only during the player's turn. Green above 50%, yellow above 25%, red below, using the colors already in the UI. The pads did not move.
- **Timeout:** at zero the game ends immediately with "TIME'S UP! GAME OVER". A click that arrives after time ran out is ignored.
- **Reset:** a new round or pressing R resets the timer; it stays hidden during playback.

Final-pass fix

My earlier click-highlight fix had a side effect: when you finished a pattern, the next round started instantly, wiping your last click's highlight and playing its tone on top of the new round's first flash. I replaced it with a short 600 ms pause before each round's playback. The highlight fades naturally, the tones no longer overlap, and the earlier highlight-clearing code is removed.

Code changes

Diff shown against the file you uploaded in this turn.

game/game_engine.py		+79 -10
@@ -19,6 +19,15 @@		
TONE_FREQUENCIES = [262, 330, 392, 523]		
TONE_LENGTH_MS = 200		
+ # Player turn timer (milliseconds): total time allowed to enter the WHOLE pattern.		
+ # Longer patterns get more time: round 1 = 4 s, round 5 = 8 s, round 10 = 13 s.		
+TIMER_BASE_MS = 3000		
+TIMER_PER_STEP_MS = 1000		
+		
+ # Short breather before each round's playback so the player's last click (and its		
+ # tone) isn't mixed with the first flash of the new pattern.		
+ROUND_START_DELAY_MS = 600		
+		
class GameEngine:		
def __init__(self, width, height):		
@@ -51,6 +60,11 @@		
self.player_lit_button = None		
self.player_lit_start = 0		
self.player_flash_duration = 150		
+		
+ self.turn_start_time = 0		
+ self.turn_time_limit = TIMER_BASE_MS		
+ self.turn_time_left = TIMER_BASE_MS		
+ self.game_over_reason = ""		
self.font_title = pygame.font.SysFont(None, 40)		
self.font_medium = pygame.font.SysFont(None, 28)		
@@ -93,6 +107,20 @@		
self.flash_duration = max(MIN_FLASH_MS, BASE_FLASH_MS - FLASH_STEP_MS * rounds_played)		
self.pause_duration = max(MIN_PAUSE_MS, BASE_PAUSE_MS - PAUSE_STEP_MS * rounds_played)		
+ def get_turn_time_limit(self):		
+ """Time allowed for the player's turn, scaled by the pattern length."""		
+ return TIMER_BASE_MS + TIMER_PER_STEP_MS * len(self.sequence)		
+		
+ def check_turn_timeout(self, now):		
+ """Update the remaining time; end the game if it ran out. True on timeout."""		
+ elapsed = now - self.turn_start_time		
+ self.turn_time_left = max(0, self.turn_time_limit - elapsed)		
+ if self.turn_time_left <= 0:		
+ self.game_over_reason = "TIME'S UP! GAME OVER"		
+ self.state = "GAME_OVER"		
+ return True		
+ return False		
+		
def start_next_round(self):		
new_color = random.randint(0, 3)		
@@ -101,18 +129,15 @@		
self.update_playback_speed()		
- # Drop any leftover click highlight so it can't overlap/cut the first flash		
- if self.player_lit_button is not None:		
- self.player_lit_button.is_lit = False		
- self.player_lit_button = None		
-		
self.player_input.clear()		
self.state = "WATCH"		
- self.showing_step = 0		
+ self.showing_step = -1 # -1 = short delay before the first flash		
self.step_start_time = pygame.time.get_ticks()		
- self.is_flashing = True		
- self.buttons[self.sequence[0]].is_lit = True		
- self.play_tone(self.sequence[0])		

```

+         self.is_flashing = False
+
+         # Timer is reset here but only starts counting once the player's turn begins
+         self.turn_time_limit = self.get_turn_time_limit()
+         self.turn_time_left = self.turn_time_limit
+
+     def update(self):
+         now = pygame.time.get_ticks()
@@ -123,6 +148,16 @@
+         self.player_lit_button = None
+         if self.state == "WATCH":
+             if self.showing_step < 0:
+                 if now - self.step_start_time >= ROUND_START_DELAY_MS:
+                     self.showing_step = 0
+                     first_id = self.sequence[0]
+                     self.buttons[first_id].is_lit = True
+                     self.play_tone(first_id)
+                     self.is_flashing = True
+                     self.step_start_time = now
+             return
+
+         current_btn_id = self.sequence[self.showing_step]
+         if self.is_flashing:
@@ -141,6 +176,11 @@
+             self.step_start_time = now
+         else:
+             self.state = "PLAYER_TURN"
+             self.turn_start_time = now
+             self.turn_time_left = self.turn_time_limit
+
+         elif self.state == "PLAYER_TURN":
+             self.check_turn_timeout(now)
+
+     def handle_event(self, event):
+         if self.state == "GAME_OVER":
@@ -149,6 +189,10 @@
+         return
+         if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
+             # A click that arrives after time ran out must not count
+             if self.check_turn_timeout(pygame.time.get_ticks()):
+                 return
+
+         for btn in self.buttons:
+             if btn.contains(event.pos):
+                 if self.player_lit_button is not None:
@@ -166,6 +210,7 @@
+             current_idx = len(self.player_input) - 1
+             if self.player_input[current_idx] != self.sequence[current_idx]:
+                 self.game_over_reason = "WRONG PATTERN! GAME OVER"
+                 self.state = "GAME_OVER"
+                 return
+
@@ -177,11 +222,32 @@
+         self.sequence.clear()
+         self.player_input.clear()
+         self.score = 0
+
+         self.game_over_reason = ""
+         for btn in self.buttons:
+             btn.is_lit = False
+             self.player_lit_button = None
+             self.start_next_round()
+
+     def render_timer_bar(self, screen):
+         left = self.buttons[0].rect.left
+         width = self.buttons[1].rect.right - left
+         bar = pygame.Rect(left, 125, width, 14)
+
+         ratio = self.turn_time_left / self.turn_time_limit

```

```

+         if ratio > 0.5:
+             fill_color = (80, 240, 130)
+         elif ratio > 0.25:
+             fill_color = (255, 220, 80)
+         else:
+             fill_color = (240, 70, 70)
+
+         pygame.draw.rect(screen, (45, 48, 58), bar, border_radius=7)
+         fill = bar.copy()
+         fill.width = int(bar.width * ratio)
+         if fill.width > 0:
+             pygame.draw.rect(screen, fill_color, fill, border_radius=7)
+         pygame.draw.rect(screen, (70, 75, 85), bar, width=2, border_radius=7)
+
+     def render(self, screen):
+         screen.fill((22, 24, 30))
@@ -196,6 +262,9 @@
+         status_surf = self.font_medium.render(status_text, True, status_color)
+         screen.blit(status_surf, (self.width // 2 - status_surf.get_width() // 2, 95))
+
+         if self.state == "PLAYER_TURN":
+             self.render_timer_bar(screen)
+
+         for btn in self.buttons:
+             btn.render(screen)
@@ -204,7 +273,7 @@
+         overlay.fill((0, 0, 0, 200))
+         screen.blit(overlay, (0, 0))
-         over_surf = self.font_title.render("WRONG PATTERN! GAME OVER", True, (240, 70, 70))
+         over_surf = self.font_title.render(self.game_over_reason, True, (240, 70, 70))
+         screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 40))
+         final_score_surf = self.font_medium.render(f"Final Score: {self.score}", True, (255, 255, 255))

```

Testing performed

- Time limit scales with pattern length; timer resets every round and after R; bar hidden during playback.
- A click 1 ms before the deadline counts; a late click is ignored and ends the game with the timeout message.
- Partial entry followed by timeout, wrong click, and R restart all behave correctly.
- Rapid clicks never leave a pad stuck lit; clicks during playback are ignored.
- A random fuzz run (30 seeds × 4,000 steps of random clicks, key presses and time jumps) never broke a state or timer rule.
- No-audio fallback still works; main.py starts cleanly.
- Rendered the bar at full, mid, low and expired to check the look (below).

Timer bar at full, mid, low, and expired

